Running the eDNAVisualJointModel re-runs on stats3

EJ45

2026-10-25

Quick orientation

These are the re-runs that the move to the 35°N study domain needed (ROADMAP_2026-10-06, 6.0; the remaining ones are in ROADMAP_2026-10-21, section 3). As with the FinWhale pipeline, the cleanest set-up is **SSH + tmux**: every job is a non-interactive `Rscript` that writes files.

> **Now (2026-10-28): re-fit the real eDNA-only and joint models.** The first real fits (stage 6) ended, but the eDNA-only and joint fits did not converge: the hake qPCR noise terms mixed badly (max R-hat 1.25 and 2.03; the visual-only fit is clean). Two new jobs re-fit them with the Ct SD bounded at low copies (Part 6, "Getting ready"): 8 cores, about a day and a half. Seed 1 needs no more runs (Part 6). **The tier-1 replicates (seeds 2–5) wait until later this week**; their steps are in Part 6, "The tier-1 replicates". Everything else in this guide is the earlier set-up and still applies.
>
> **Also from 2026-10-28: maps and predictive checks** (Part 6, "Maps and predictive checks"). These jobs run from a **second checkout**, so they don't recompile the model the running re-fits use.

stats3 specs (from the IT FAQ): Intel i9-13900K with 24 physical cores and 32 hyperthreads, 128 GB RAM, RAID-1 NVMe (`/fast_raid`) and HDD (`/local_raid`).

Each Stan chain uses one core, and a fit runs 2–4 chains. No fit uses within-chain threading, so a single fit never uses more than 4 cores. Memory is not a constraint (the largest arrays are tens of MB per fit), so **cores are the budget: keep the total at or below 24.** Hyperthreads add little for Stan.

One script, `tools/stats3_reruns.sh`, runs everything. It groups the re-runs into five **stages**, with one tmux window and one log (`logs/<job>.log`) per job. Every job runs under `nice` and stops at its first failing step.

Stage	Jobs	Cores	Rough time
1	`prep`: real data (`01_load_muri.r`, `02_check_muri.r`), smoke test, `tests/visual_detection.R`, compile every model	2	about 1 h
2	HSGP4eDNA 2-D validation (4 jobs), visual validation (normal and sea state), joint simulator check	23	hours; `hsgp_bathysp_depthspref` 1–2 days
3	Joint comparison: modes `visual`, `edna`, `joint`, `joint_conv` (4 chains each) for every seed in `CMP_SEEDS` (default `"2 3"`); with `CMP_SEA=1` the sea-state version (the replicates; Part 6)	16 per seed (+12 for the without-Beaufort contrast)	about 6–8 h each (`visual` about 1 h)
4	Explainer fits: simulated eDNA and visual, real visual (budget 800 and 1600, both species)	14	about 1–3 h each
5	`finish`: comparison tables and `joint_compare_s<seed>.png` for every seed with fits, the remaining explainer figures	1	minutes

The real eDNA explainer fit (`expl_edna_real`) is in no stage. Its data are

in (Ct per well, the hake standards, volumes, dilutions and target depths; ROADMAP_2026-10-21, section 5) and the joint model can use them, but the job's script (`docs/explainer/04_fit_edna_real.R`) still runs the earlier metabarcoding-only fit. Don't launch it until that script is rewritten (ROADMAP_2026-10-21, section 5 item 7).

The times are estimates. The budget is now 800 functions and the domain is larger, which roughly doubles the old run times. `bathysp_depthspref` (1185 bottles) took 15 h on the old set-up. The whole list is about 280 core-hours, or about 2 days of wall time on stats3.

Stage 1 must finish first, because it checks the real data and compiles every Stan model. Stages 2 to 4 each need only stage 1, and stage 5 needs 3 and 4.

Part 1: Connect to stats3 (every session)

This is the same as for FinWhale. From a Mac terminal:

```
ssh -J ej45@jump.st-andrews.ac.uk ej45@maths-stats3.st-andrews.ac.uk
```

With the `~/.ssh/config` block from the FinWhale guide, this is just `ssh stats3`. Work on the fast disk:

```
cd /fast_raid/ej45
```

Part 2: SSH keys (optional, one time)

If you already set up keys and the `jump` / `stats3` hosts for FinWhale, skip this. Otherwise, follow Part 2 of the FinWhale guide:

- `ssh-keygen`;
- `ssh-copy-id` to both hops;
- the `Host jump` / `Host stats3` block in `~/.ssh/config`.

The `scp` and `rsync` commands below use the `stats3` alias. Without it, give the full host name and the jump host:

```
scp -J ej45@jump.st-andrews.ac.uk FILES \
    ej45@maths-stats3.st-andrews.ac.uk:/fast_raid/ej45/...
rsync -av -e 'ssh -J ej45@jump.st-andrews.ac.uk' \
    ej45@maths-stats3.st-andrews.ac.uk:/fast_raid/ej45/... DEST/
```

Part 3: Clone the repo and its submodule (one time)

On stats3:

```
cd /fast_raid/ej45
git clone --recurse-submodules https://github.com/MMARINeDNA/eDNAVisualJointModel.git
cd eDNAVisualJointModel
git submodule status        # external/HSGP4eDNA should show v1.5-48-g5515a04 (v1.12)
```

eDNAVisualJointModel is public, but its submodule **HSGP4eDNA is private**. So the clone asks for a GitHub username and a **personal access token** (not your password). Create one at github.com → Settings → Developer settings → Personal access tokens:

- a fine-grained token: set *Resource owner* to `MMARINeDNA` and give it read access to `Contents`;
- or a classic token with the `repo` scope.

To avoid pasting the token on every pull, keep it in memory for a day:

```
git config --global credential.helper 'cache --timeout=86400'
```

To update later: `git pull && git submodule update --init`.

Part 4: Copy the private input data (only for `prep` and real-data jobs)

The raw MURI files are **not in the repo**, because the repo is public, and `.gitignore` keeps them out. Only the real-data loader (`R/01_load_muri.r`) and its check (`R/02_check_muri.r`) read them, so only `prep` (stage 1) needs them. Everything else, the replicates included, uses simulations or the tracked, derived `data/processed/muri_{edna,visual}.rds` and `qpcr_standard_curves.rds`.

The loader now reads four private files (from 2026-10-21): the qPCR lab's `qPCR_2019A.rds`, the compiled metabarcoding table `M3_compiled_taxon_table_wide.csv`, the sample metadata `Hake_2019_metadata.csv`, and `SITEINFO_CCE_2018_Pp_Lo_Mn.csv` (perpendicular distances). `MV1_MURI_df.csv` is no longer used. From a **Mac terminal in your main checkout**, which has them:

```
cd /Users/ej45/Documents/MMARINeDNA/eDNAVisualJointModel
scp data/qPCR_2019A.rds data/M3_compiled_taxon_table_wide.csv \
    data/Hake_2019_metadata.csv data/SITEINFO_CCE_2018_Pp_Lo_Mn.csv \
    stats3:/fast_raid/ej45/eDNAVisualJointModel/data/
```

Then check on stats3 with `ls -lh /fast_raid/ej45/eDNAVisualJointModel/data/`. If a file is missing, the `prep` job stops with a message pointing here.

Bathymetry. On its first run, the loader downloads ETOPO 2022 bathymetry from NOAA (via `marmap`, a few MB) and caches it as `data/processed/etopo2022_60s_cce.rds`. If stats3 cannot reach NOAA, copy the cache across from a checkout that has one. The `stan-data-formatting` worktree does:

```
cd /Users/ej45/Documents/MMARINeDNA/eDNAVisualJointModel/.claude/worktrees
scp stan-data-formatting-61936f/data/processed/etopo2022_60s_cce.rds \
    stats3:/fast_raid/ej45/eDNAVisualJointModel/data/processed/
```

Part 5: R packages and CmdStan (one time)

Check R with `R --version`. The Mac runs R 4.5.2; 4.3 or later should be fine. Then:

```
cd /fast_raid/ej45/eDNAVisualJointModel
R
```

In R:

```
install.packages(c(
  "tidyverse", "posterior", "patchwork", "viridis", "bayesplot", "loo",
  "gridExtra", "data.table", "jsonlite", "sf", "terra", "marmap"
))
install.packages("cmdstanr",
  repos = c("https://stan-dev.r-universe.dev", getOption("repos")))

cmdstanr::check_cmdstan_toolchain(fix = TRUE)
cmdstanr::install_cmdstan(dir = "/fast_raid/ej45/cmdstan",
                          version = "2.38.0", cores = 8)   # same CmdStan as the Mac
cmdstanr::cmdstan_path()        # copy this path
quit(save = "no")
```

Packages install into `/local_raid/ej45/R/library` through the `.Rprofile` IT set up. `sf` and `terra` build against GDAL, GEOS and PROJ; if they fail with a missing-library error, ask maths-tech-support. The CmdStan build takes 5–10 minutes.

Point cmdstanr at CmdStan for every session (use the path printed above):

```
echo 'CMDSTAN=/fast_raid/ej45/cmdstan/cmdstan-2.38.0' >> ~/.Renviron
Rscript -e 'cmdstanr::cmdstan_version()'    # should print 2.38.0
```

Part 6: Run the re-runs inside tmux

The script creates the tmux session (`reruns`) itself. Start with the overview, then stage 1:

```
cd /fast_raid/ej45/eDNAVisualJointModel
tools/stats3_reruns.sh list
tools/stats3_reruns.sh stage 1
tools/stats3_reruns.sh status       # repeat until prep says DONE (about 1 h)
```

`prep` reruns the loader, so the log should say each processed object **matches the committed copy**. If one differs, stop and look before going on. The smoke test's 44 checks must pass. If `prep` says `FAILED`, read `logs/prep.log`. Simulation-only work (the replicates below) can skip `prep` and run the much shorter `compile` job instead.

Getting ready (now)

Steps 1–3 below (stop old jobs, update the checkout, `compile`) are needed before **any** of the new jobs. On 2026-10-28 the branch is `claude/real-refit` until it is merged; after that, `main`. Then, once:

```
# set stats3's seed-1 fits aside (they are on a different dataset from the
# Mac's; see below) and the earlier plain-comparison runs
mv outputs/joint/compare_sea_s1 outputs/joint/compare_sea_s1_stats3
mkdir -p outputs/joint/old && mv outputs/joint/compare_s2 outputs/joint/compare_s3 outputs/j
```

and launch the two real-data re-fits (8 cores; see "The real-data fits"):

```
tools/stats3_reruns.sh launch jreal_edna_ctcond
tools/stats3_reruns.sh launch jreal_joint_conv_ctcond
tools/stats3_reruns.sh status
```

The seed-1 demonstration is done, on the Mac. The same seed gives a different dataset on stats3 than on the Mac: the simulator draws the GP fields with `MASS::mvrnorm()`, which goes through `eigen()`, and its output depends on the machine's LAPACK library. So stats3's joint and free-conversion fits of 2026-10-27 are on another dataset from the Mac's eDNA-only and visual-only fits, and the four can't go in one table. The Mac's four fits (`outputs/joint/compare_sea_s1/`, all on one dataset and all converged) are the demonstration. Its joint fits predate the bound on the NB parameter, which matters only for the real data. stats3's two fits are kept as `compare_sea_s1_stats3/`. `R/compare_joint.r` now refuses to fit a mode whose dataset differs from the `sim.rds` already in its folder, and `SIM_FILE=<path>` fits a saved dataset instead of simulating one.

Steps 4–6 are for the replicates, later this week. Every mode of a seed must run on the same machine.

Maps and predictive checks (2026-10-28)

The joint model now puts the field on a prediction grid and keeps predictive-check summaries (README, "Joint model"). Three sets of jobs use them:

Job	What	Cores, time
`cmpgrid_s1_<mode>` (`edna`, `visual`, `joint`, `joint_conv`, `prior`)	seed 1 refitted on the laptop's dataset with the grid and checks: the 12-panel maps (truth, eDNA only, visual only, joint) and the predictive checks	4 each; visual ~3 h, prior a few hours, the others ~1 day

Job	What	Cores, time
`jreal_visual_b1`, `jreal_prior`	the real visual-only fit with the hazard-rate shape bounded at b >= 1 (Len), with the real maps and checks; the real prior predictive	4 each; ~21 h, a few hours
`jgq_<mode>` (`edna`, `joint_conv`)	the grid and checks added to the finished `jreal_<mode>_ctcond` fits from their chain CSVs, without refitting	1 each, minutes

1. A second checkout (one time). The `_ctcond` re-fits are running the compiled model in the main checkout; recompiling it there could fail ("text file busy") or disturb them. After the PR is merged into `main`:

```
cd /fast_raid/ej45
git clone --recurse-submodules https://github.com/MMARINeDNA/eDNAVisualJointModel.git eDNAVi
cd eDNAVisualJointModel_grid
tools/stats3_reruns.sh run compile            # ends with "DONE compile"
mkdir -p outputs/joint
```

2. Copy the laptop's seed-1 dataset (the same seed simulates different data on stats3; Part 6, "Getting ready"). On the Mac:

```
scp /Users/ej45/Documents/MMARINeDNA/eDNAVisualJointModel/.claude/worktrees/ednavisualjointm
```

3. Launch, from the second checkout, in their own tmux session (16 cores, plus the re-fits' 8):

```
cd /fast_raid/ej45/eDNAVisualJointModel_grid
export SESSION=grid
for m in edna visual joint; do tools/stats3_reruns.sh launch cmpgrid_s1_$m; done
tools/stats3_reruns.sh launch jreal_visual_b1
tools/stats3_reruns.sh status
```

When `cmpgrid_s1_visual` ends (a few hours), launch `cmpgrid_s1_prior` and `jreal_prior` in its place; `cmpgrid_s1_joint_conv` is optional (its maps match the joint fit's) and can wait for free cores. Check on them with `tmux attach -t grid`.

4. When the `_ctcond` re-fits end, in the main checkout:

```
cd /fast_raid/ej45/eDNAVisualJointModel
git checkout main && git pull && git submodule update --init
tools/stats3_reruns.sh run compile
tools/stats3_reruns.sh run jgq_edna && tools/stats3_reruns.sh run jgq_joint_conv
```

Pull everything back with Part 9.

The tier-1 replicates (later this week)

The sea-state joint comparison for seeds 1–5: each seed simulated with segment Beaufort and fitted eDNA-only, visual-only, joint and joint with free conversion factors, all with the Beaufort covariate; seeds 1 and 2 also without it, for contrast. The fits use the settings that are now `R/compare_joint.r`'s defaults: negative-binomial segment counts for both cetaceans, non-centred junk noise and `adapt_delta = 0.9`, with the NB parameter bounded (`inv_sqrt_phi_nb >= 0.01`). Seed 1 with the covariate is done first (above); the rest are 22 jobs of 4 chains each.

1. Connect and stop anything left from an earlier launch. If yesterday's stage-3 jobs are running, they use the model without the NB bound and can get stuck (Part 8):

```
ssh stats3
cd /fast_raid/ej45/eDNAVisualJointModel
tools/stats3_reruns.sh status
tmux kill-session -t reruns          # only if jobs from an earlier launch are running
```

2. Update the checkout to the branch with the replicate settings (until it is merged into `main`) and the HSGP4eDNA v1.12 submodule:

```
git fetch
git checkout claude/replicates
git pull
git submodule update --init
git log --oneline -3                 # the latest should be the guide update of 2026-10-28 or
                                     # (it must include the sim.rds check in R/compare_joint
git submodule status                 # v1.5-48-g5515a04
```

3. Compile the models once (a few minutes; the Stan files changed). Without this, every job of a wave would compile the joint model at the same moment. Everything on the branch was checked on the Mac on 2026-10-26: the smoke test, the exactness tests, and each job type (`cmpsea_*`, `cmpseanobft_*`, `jreal_*`) run through this script with tiny chains.

```
tools/stats3_reruns.sh run compile   # ends with "DONE compile"
```

4. Launch the jobs in waves, keeping to about 24 cores. A seed's four jobs use 16 cores; its visual-only job ends after about an hour, the other three take about 6–8 hours. Check `uptime` before each wave and start the next one when the previous wave's long jobs end.

Wave	Command	Jobs	Cores
1	`CMP_SEA=1 CMP_NOBFT=0 CMP_SEEDS="2 3" tools/stats3_reruns.sh stage 3`	seeds 2–3, with the covariate	32 for the first hour, then 24
2	`CMP_SEA=1 CMP_NOBFT=0 CMP_SEEDS="4 5" tools/stats3_reruns.sh stage 3`	seeds 4–5	32, then 24
3	`for s in 2 3; do for m in visual joint joint_conv; do tools/stats3_reruns.sh launch cmpseanobft_s${s}_$m; done; done`	seeds 2–3 without the covariate	24, then 16

Seed 1 is done on the Mac, so it is not in the waves. The contrast without the covariate uses seeds 2–3 instead of 1–2, because it must be on the same datasets as the fits with the covariate, and those are on stats3. That is about a day in all. Results go to `outputs/joint/compare_sea_s<seed>/` and `compare_sea_nobft_s<seed>/` (one `fit_<mode>.rds` per job).

5. Check on them (Part 7). Each job's log ends with a line like `joint: 465 min | divergences 0 | treedepth 0 | max Rhat 1.06`. A visual-only job still in warmup after about 2 hours is the stuck-chain problem (Part 8).

6. Pull the results back when all 22 are done (Part 9, the replicates block) and ask Claude for the summary table across seeds.

The real-data fits (stage 6, 2026-10-26; re-fits 2026-10-28)

The joint model on the real 2019 eDNA and 2018 visual data (`R/fit_joint_real.r`; ROADMAP_2026-10-21, section 5 item 7). Three jobs of 4 chains, 1000 warmup and 1000 draws each:

Job	Fit
`jreal_edna`	eDNA only: the real eDNA fit, with Ct and per-plate qPCR curves estimated from the 659 standards, the dilution offsets, per-bottle volumes and the sampling-depth smooth
`jreal_visual`	visual only, on the same basis and priors (humpback hazard-rate + group size; PWSD half-normal + group size + Beaufort + visibility; NB counts for both)
`jreal_joint_conv`	both sources, the cetaceans' eDNA conversion factors estimated (the real ones are unknown)

They read only the tracked `data/processed/` objects, so they need `compile` but not `prep` or the private data. After steps 1–3 above:

```
tools/stats3_reruns.sh stage 6
tools/stats3_reruns.sh status          # the three jreal_* jobs
```

On a busy stats3 (load 115–140) they took 37 h (`jreal_edna`, `jreal_joint_conv`) and 21 h (`jreal_visual`). Results are in `outputs/joint/real/` (`fit_<mode>.rds`, and `real_{runs,params,floor}.csv` over the modes found).

The first run (2026-10-27). No divergences in any fit. The visual-only fit converged (max R-hat 1.004). The eDNA-only and joint fits did not (1.25 and 2.03): the slow parameters are all hake's qPCR side (the Ct-noise terms `gamma0_ct_est` and `gamma1_ct_est`, the bottle noise, the depth smooth and a bathymetry term), while the cetaceans' fields, the visual parameters and the conversion factors mixed well. The Ct SD is evaluated at the expected copies, and with `gamma1_ct_est` < 0 it grows without bound as the copies fall, so a low-copy bottle can explain any Ct by a large SD, against the bottle noise. HSGP4eDNA v1.12's `ct_sd = "conditional"` bounds it at its one-copy value.

The re-fits `jreal_edna_ctcond` and `jreal_joint_conv_ctcond` use that bound (`CT_SD=conditional`), write to `outputs/joint/real_ctcond/`, and keep the CmdStan chain CSVs in `outputs/joint/real_ctcond/csv_<mode>/` on stats3, so a stuck chain can be looked at. Each fit's `.rds` now also holds every chain's mean of each parameter (`chain_means`), and the log prints them for any parameter with R-hat above 1.05. Expect about a day and a half each.

Other stage-3 runs

The plain (calm-sea) comparison, as before:

For the presentation, launch the joint comparison next. With the default `CMP_SEEDS="2 3"` it runs 8 jobs (32 chains for the first half hour, 24 once the two visual-only fits end), and should finish overnight:

```
tools/stats3_reruns.sh stage 3                      # seeds 2 and 3
CMP_SEEDS="4 5" tools/stats3_reruns.sh stage 3      # second wave, once the first is done
CMP_SEEDS="1" tools/stats3_reruns.sh stage 3        # seed 1 (needs the data-based start)
```

Sea-state comparison: `CMP_SEA=1` runs the four modes with the Beaufort covariate and, unless `CMP_NOBFT=0`, the three visual modes without it (28 cores per seed); the replicates above use it.

A single fit can be relaunched by name, e.g. `tools/stats3_reruns.sh launch cmp_s3_joint`. The tables and figure for a seed are rebuilt by `finish` (stage 5); for the presentation it is quicker to pull `outputs/joint/` back to the Mac (Part 9) and build them there.

The rest, after the presentation. Stage 2 is the full validation; it takes the longest:

```
tools/stats3_reruns.sh stage 2
```

It uses 23 cores. Its visual and joint-simulator jobs finish within hours, as do three of the four HSGP4eDNA jobs; `hsgp_bathysp_depthspref` (3 cores) runs for 1–2 days. Launch stage 4 when about 14 cores are free, and stage 5 once stages 3 and 4 are done:

```
tools/stats3_reruns.sh stage 4
tools/stats3_reruns.sh stage 5
```

Before launching a stage, check `uptime` (load average) or `htop`. Hold the stage back if other users are busy.

To watch a job: `tmux attach -t reruns`, then `Ctrl-b w` to pick a window. To detach, press `Ctrl-b d`; the jobs keep running after you log out. A finished job's window stays open at a shell prompt.

To re-run one job, e.g. after a failure:

```
tools/stats3_reruns.sh launch hsgp_bathygp_surface
```

What each job writes (git-ignored unless marked *tracked*):

Job	Output
`prep`	`data/processed/muri_{edna,visual}.rds` (*tracked*; should be unchanged), `outputs/smoke/`
`hsgp_*`	`external/HSGP4eDNA/outputs/<scenario>/` (fit summary, recovery CSV and PDF)

Job	Output
`visual_val`, `visual_val_sea`	`outputs/visual/validation/`, `outputs/visual/validation_seastate/`
`joint_sim`	`outputs/joint_sim/validation/`
`cmp_s<seed>_<mode>`	`outputs/joint/compare_s<seed>/fit_<mode>.rds`
`expl_*`	`outputs/explainer/`, `docs/figures/*.png` (*tracked*)
`finish`	`outputs/joint/compare_s<seed>/compare_*.csv`, `docs/figures/joint_compare_s<seed>.png` and the remaining explainer figures

Part 7: Check progress

From the Mac, without attaching:

```
ssh stats3 'cd /fast_raid/ej45/eDNAVisualJointModel && tools/stats3_reruns.sh status'
ssh stats3 'tail -n 30 /fast_raid/ej45/eDNAVisualJointModel/logs/visual_val.log'
```

Stan writes each chain's progress to the log (`Chain 2 Iteration: 400 / 1600`). The fit scripts print divergences, max Rhat and the run time when a fit ends, and warn when Rhat is above 1.05.

Part 8: If something goes wrong

- **A job says `FAILED`.** The end of its log has the R error. Fix the cause, then run `launch <job>` again. Other jobs carry on.
- **A model compile error in a stage 2–4 job** means `prep` did not finish. Run it first.
- **`tests/visual_detection.R` recompiles the visual model.** It runs inside `prep`, so don't relaunch `prep` while visual fits are running.
- **Stopping jobs.** To stop one, attach, pick its window and press `Ctrl-c`. To stop everything, run `tmux kill-session -t reruns`.
- **A fit stuck in warmup for hours** (no `Iteration: ... (Sampling)` lines; seen on 2026-10-24 in a visual-only fit) was the NB parameter running off to 0 on near-Poisson sightings. The bound in commit `e0ed941` fixes it; check `git log` shows it, run `compile`, and relaunch the job by name.
- **Two jobs compiling at once** can leave a broken executable. Run `tools/stats3_reruns.sh run compile` before a wave whenever the Stan files have changed.

Part 9: Pull results back to the Mac

Leave the CmdStan chain CSVs on stats3. They can be about 330 MB per chain, and the fits keep what they need in `.rds` summaries. In your main checkout, on an up-to-date `main` with a clean tree:

```
cd /Users/ej45/Documents/MMARINeDNA/eDNAVisualJointModel
git switch main && git pull && git submodule update --init
R=stats3:/fast_raid/ej45/eDNAVisualJointModel
X="--exclude=*-202[0-9]*-[0-9]-*.csv"      # CmdStan chain CSVs

rsync -av "$X" $R/outputs/ outputs/
rsync -av "$X" $R/external/HSGP4eDNA/outputs/ external/HSGP4eDNA/outputs/
rsync -av $R/docs/figures/ docs/figures/
rsync -av $R/logs/ logs/
```

For the replicates, only the joint comparison is needed, which is small (summaries only). Into the session's worktree or any checkout:

```
rsync -av "$X" $R/outputs/joint/ outputs/joint/
rsync -av $R/logs/ logs/
```

For the real-data re-fits, leave their chain CSVs on stats3:

```
rsync -av "$X" --exclude='csv_*/' $R/outputs/joint/real_ctcond/ outputs/joint/real_ctcond/
rsync -av $R/logs/ logs/
```

For the maps and predictive checks (the second checkout):

```
G=stats3:/fast_raid/ej45/eDNAVisualJointModel_grid
rsync -av "$X" --exclude='csv_*/' $G/outputs/joint/compare_sea_s1_grid/ outputs/joint/compar
rsync -av "$X" --exclude='csv_*/' $G/outputs/joint/real_b1/ outputs/joint/real_b1/
rsync -av $G/logs/ logs/
```

Seed 1 is the Mac's `outputs/joint/compare_sea_s1/` (Part 6). Pull only after moving stats3's copy aside (Part 6, "Getting ready"), or the pull overwrites the Mac's joint fits with fits to another dataset.

`docs/figures/*.png` then shows up as changed in `git status`. `data/processed/` is not copied back, because `prep` checked that it matches `main`.

Then ask Claude to do three things:

- rebuild the tables: the HSGP4eDNA README §4 (and a v1.5.x release), this README's results and ROADMAP_2026-10-21;
- rewrite the explainer's numbers, and re-render it locally with `quarto render docs/explainer.md --to html --embed-resources`;
- open the PRs.

TL;DR for repeat sessions

```
ssh stats3
cd /fast_raid/ej45/eDNAVisualJointModel
tools/stats3_reruns.sh status        # what is done, running or failed
tools/stats3_reruns.sh run compile   # after a git pull that changed Stan files
tools/stats3_reruns.sh stage <n>     # launch the next stage (replicates: Part 6 waves; rea
```

```
tmux attach -t reruns                # Ctrl-b w to pick a window, Ctrl-b d to detach
exit
```

Then the `rsync` block in Part 9 from the Mac.